Class 32: Continuous Monitoring and Scalability

Session 32 of 36 – MLOps Block

Data-driven Systems Engineering

Agenda

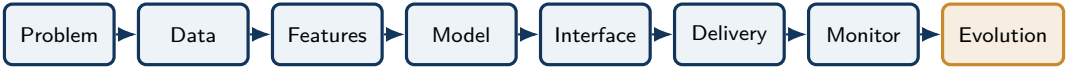
- Why monitoring matters
- Monitoring as risk control, and monitoring types
- Data drift versus concept drift, and detection strategies
- Response after alerts
- Scalability: load, horizontal scaling, and cost-latency trade-offs

Monitoring and scaling turn deployment into an ongoing job

Learning goals

- Distinguish system, data, model, and business monitoring layers.
- Compare data drift, concept drift, and pipeline failure symptoms.
- Design practical responses after a drift alert.
- Reason about load, horizontal scaling, and cost-latency trade-offs for serving ML systems.

Course Map and Running Cases



Today in the course

Focus: Monitoring, drift response, and scaling under real load.
Bridge: This class extends delivery into sustained operation: a shipped system must stay correct as data drifts and stay responsive as traffic grows.

Fraud case

In the fraud case, monitoring covers data shift, score behavior, and analyst workload, while scaling covers what happens to latency during a transaction spike.

Pasteurization case

In the pasteurization case, monitoring covers sensor quality and state transitions, while scaling covers serving many simultaneous plant dashboards from one stream.

Course thread: Observation and scale are where an engineered system proves itself over time; the course now carries this into project execution.

Why monitoring matters

- ML systems are data products, not static binaries.
- Degradation can be silent and expensive.
- Labels may arrive late, so proxies matter.
- Monitoring is evidence for operations, governance, and improvement.

Monitoring is quantified risk management

- Data risk: changed distributions, missing features, schema violations.
- Model risk: degraded performance, instability, concept drift, unfairness.
- Operational risk: broken pipelines, latency spikes, failed endpoints.
- Business risk: review overload, poor intervention yield, loss of trust.

Meaning

Monitoring turns hidden failure into visible evidence that teams can act on.

What should be monitored in an ML service?

System layer: latency, errors, uptime, resources

Data layer: schema, ranges, drift, missingness

Model layer: scores, errors, calibration

Business layer: review load, outcomes, overrides

Data drift vs concept drift

- Data drift: input distribution changes, even if the relationship still holds.
- Concept drift: the mapping from input to target changes.
- Pipeline bugs can mimic both and must be ruled out early.

Examples

Holiday shopping patterns can cause fraud feature drift; a new attack strategy can create concept drift.

What can look like drift but is not

- A broken ingestion job that changes missing-value rates.
- A schema change that reorders or renames columns.
- A deployment mismatch between training preprocessing and live preprocessing.
- A monitoring bug that changes the way metrics are computed.

Rule

Before retraining, verify that the problem is real and not a pipeline defect.

Detection strategies

- Batch statistical tests: KS, PSI, chi-square, Jensen-Shannon variants.
- Prediction monitoring: score distribution and confidence shifts.
- Label-based metrics when ground truth arrives.
- Streaming detectors: ADWIN, Page-Hinkley, DDM, EDDM.

A practical drift-detection taxonomy

Signal type	Typical use
Feature distribution tests	Early warning when labels are delayed.
Prediction distribution checks	Detect score collapse or threshold instability.
Label-based performance	Confirm concept drift when outcomes arrive.
Streaming detectors	Catch online change in error or metric sequences.
Segment analysis	Reveal that drift affects only some populations.

Repository-style online monitoring loop

```
y_pred = model.predict_one(x)
model.learn_one(x, y)
mae.update(y, y_pred)
error = abs(y - (y_pred or 0))
adwin.update(error)
drift_flag = adwin.drift_detected
```

- ‘streamlit/pages/App_Stream.py‘ uses this pattern to combine incremental learning, error tracking, and ADWIN-based change detection.
- It is useful because the alert is tied to live behavior, not only offline charts.

Repo examples for a stronger lecture

Repo activity

- ‘streamlit/pages/App_Stream.py‘ already combines online learning with ADWIN.
- ‘documents/Forecasting_and_Drift_Pasteurization.pdf‘ defines a project around forecasting and drift detection.
- ‘pasteurization/synth_sensors.py‘ gives state-aware signals for meaningful monitoring discussions.
- ‘pasteurization/serving.py‘ is also our only in-repo example of a streaming service under simulated load.

After detection: diagnose, then respond

1. Verify the alert is not caused by ingestion or schema issues.
2. Diagnose whether the change is temporary, seasonal, or structural.
3. Choose a response: retrain, recalibrate, switch model, roll back, add human review, or change thresholds.
4. Record the event for later audit and process learning.

Performance over time and business metrics

- Model metrics: accuracy, precision, recall, AUC, MAE, RMSE.
- Operational metrics: latency, queue length, failed predictions.
- Business metrics: intervention yield, review load, false alert fatigue.
- Human signals: overrides, complaints, and trust breakdowns.

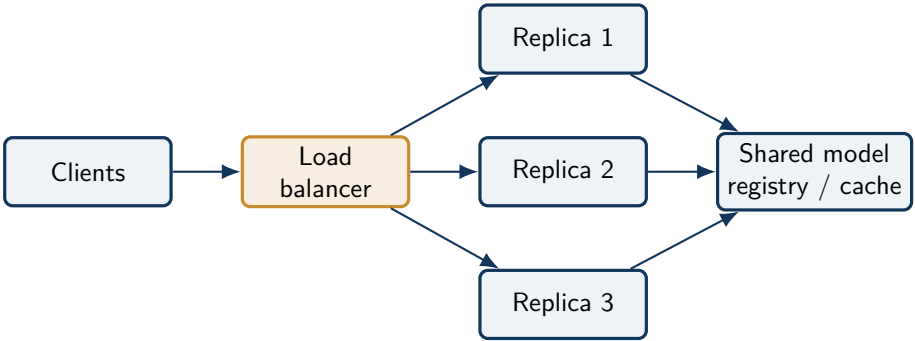
Champion-challenger and continuous comparison

- A champion model serves production traffic.
- A challenger runs in shadow mode or on sampled traffic.
- Comparison over time helps avoid switching on the basis of one lucky offline result.
- This strategy is especially useful when concept drift is suspected but labels are delayed.

From watching the system to sizing it

- Monitoring tells us the system is behaving correctly; scalability asks whether it keeps behaving correctly under more load.
- Load can grow in requests per second, batch size, number of streams, or number of simultaneous dashboards.
- A model that is accurate but too slow or too expensive under load is still a failed system.

Horizontal scaling for a serving layer



- Stateless replicas behind a load balancer are the standard pattern; each replica loads the same versioned artifact from a shared registry.
- Autoscaling adds or removes replicas based on load signals such as queue depth or CPU.

Cost-latency trade-offs when scaling ML services

Lever	Trade-off
More replicas	Lower latency under load, higher infrastructure cost.
Smaller batch size	Lower per-request latency, lower throughput per instance.
Bigger instance type	Handles larger models, higher idle-time cost.
Prediction caching	Cuts recompute for repeated inputs, risks staleness.
Async or queued inference	Smooths spikes, adds response delay.

Practical Component

Honest gap in this repository

There is currently no in-repo monitoring dashboard or load-testing exercise beyond the basic streaming demo in ‘pasteurization/serving.py‘.

- A drift-alert dashboard, or a simple load test against the Flask or streaming endpoints, does not yet exist as a repo exercise.
- This is suggested as a separate project, for example a dedicated GitHub submodule, rather than invented here.
- Until it exists, this class stays conceptual: reason about the trade-offs, sketch the architecture, but do not pretend a working benchmark is already in the repo.

Discussion prompt

Question

The fraud API is correct and well-monitored at 10 requests per second. What is the first thing likely to break at 10,000, and how would you know before it happens in production?

- This connects monitoring evidence to scaling decisions instead of treating them as separate topics.
- It also motivates load testing as a form of validation, not only a performance nice-to-have.

Papers and materials

Papers and materials

River ADWIN documentation; Practical MLOps; Reliable Machine Learning; Gama et al., A survey on concept drift adaptation; Evidently on data drift detection; Prometheus monitoring overview; The Twelve-Factor App, on stateless, horizontally scalable services.

From This Class to the Next

What stays with us

- Monitoring turns deployment into an ongoing feedback process instead of an endpoint.
- Drift detection matters only when the team has diagnosis and response mechanisms.
- Scalability is a system property to design for, not a surprise to react to after launch.

Project use

For both running cases, long-term value depends on observing data quality, model behavior, and operational health, and on knowing how the system behaves under more load than a demo ever sees.

Next connection

Class 33 turns this full lifecycle view into a working session where teams scope their own capstone project.

Course thread: Every architecture, feature, model, interface, and process choice in this course should support sustainable evolution under real, growing use.